UNIVERSIDAD DE LOS LLANOS — INGENIERÍA DE SISTEMAS

Procesos Estocásticos (Código 603504) | Proyecto Final: Reconocedor de Caracteres HMM 2D/3D

FASE 1: AUDITORÍA EXHAUSTIVA DE ARQUITECTURA Y CÓDIGO FUENTE

Inventario por carpeta y archivo, grafo de dependencias, flujos de datos y desglose de funciones con rangos de líneas verificados

1. Propósito Exacto por Carpeta y Archivo del Repositorio

El repositorio implementa una **arquitectura modular en capas con Programación Orientada a Objetos (POO)** que separa estrictamente: (a) configuración centralizada, (b) procesamiento geométrico, (c) lógica estocástica HMM pura en NumPy, (d) interfaz gráfica en tiempo real e (e) infraestructura offline de entrenamiento y evaluación.

Carpeta / Archivo	Tamaño	Propósito Exacto y Contenido Verificado
`main.py`	165 lín.	Orquestador de la aplicación de escritorio. Carga los 54 modelos JSON pre-entrenados desde `models/`, construye los extractores 2D/3D y ensambla la barra lateral junto con el `QStackedWidget` de 4 vistas. Nunca entrena ni captura datasets.
`requirements.txt`	7 lín.	Dependencias fijas para Python 3.12: `numpy==2.5.4`, `PySide6==6.12.0`, `pyqtgraph==0.14.0`, `PyOpenGL==3.1.10`, `pytest==9.1.1`. Cero librerías externas de HMM o ML.
`pytest.ini` / `.gitignore`	3 / 7 lín.	Configuración de descubrimiento de pruebas (`pythonpath = .`, `testpaths = tests`) y exclusiones de entorno virtual, cachés y datos crudos.
`README.md`	74 lín.	Guía de instalación y uso. *Nota de auditoría:* aún referencia nombres heredados (`core/topology.py`, `training/capture_tool.py`) ya fusionados o retirados en la versión actual del código.
`config/config.toml`	92 lín.	Fuente única de verdad: `[alphabet]` (27 letras minúsculas + alias `ñ="enie"`), `[features]` (`K=16`, `resample_step=0.05`, `smoothing_window=3`, `expected_letter_size=250.0`, `expected_angle_3d=0.52`, `sphere_radius=1.0`, `projection_max_symbols=45`), `[capture]`, `[evaluation]` y `[hmm.states]` (`N ∈ [4, 11]` por letra).
`core/__init__.py`	0 lín.	Marcador de paquete Python para la capa núcleo matemática.
`core/config.py`	141 lín.	Carga, valida y cachea `config/config.toml` mediante `tomllib` y dataclasses tipadas.
`core/FeatureExtractor.py`	582 lín.	Preprocesamiento geométrico, cuantización angular/saltos 2D y geodésica 3D sobre S^2, vector morfológico 7D, ray-casting y

		reconstrucción de trayectorias futuras. Sin dependencias de GUI ni HMM.
`core/HMMEngine.py`	612 lín.	Implementación 100% nativa en NumPy puro: `HMM` (Forward escalado, Backward, Viterbi log-space, Baum-Welch, penalización de duración Huberizada h_4 y proyección), inicializadores Bakis y `HMMClassifier` Bayesiano con normalización por longitud y ajuste global.
`gui/__init__.py`	5 lín.	Docstring de paquete que documenta el aislamiento de la interfaz respecto a la lógica HMM.
`gui/MainMenu.py`	198 lín.	**Ventana A:** pantalla minimalista de bienvenida con tarjetas interactivas Modo 2D y Modo 3D.
`gui/View2D.py`	369 lín.	**Ventana B:** lienzo 2D interactivo con recuadro guía 250×250 px, captura en tiempo real, botones **Deshacer trazo** y **Limpiar lienzo**, barra inferior de telemetría en vivo, proyección punteada verde y panel lateral de probabilidades.
`gui/View3D.py`	545 lín.	**Ventana C:** lienzo esférico OpenGL con recuadro guía naranja de ±30° y trazo naranja, ray-casting exacto, rotación/zoom, proyección geodésica verde y panel lateral.
`gui/RankingPanel.py`	283 lín.	Panel derecho compartido por Ventana B y C: lista ordenada de las 27 letras con glifo manuscrito, barra píldora, porcentaje estadístico normalizado y barra de desplazamiento delgada estilizada.
`gui/LetterGuide.py`	350 lín.	Vista interactiva de aprendizaje sincronizada con `canonical_glyphs.json`: muestra orden de sub-trazos (1, 2, 3), flechas direccionales, animación paso a paso y lienzo de práctica.
`data/canonical_glyphs.json`	94 KB	27 plantillas canónicas normalizadas en $[0, 1]^2$ desde los medoides modales del dataset (`a, g, k, p` en 2 trazos; `q` en 3 trazos; `x` izq→der + abajo→arriba).
`data/glifo-dataset-2026-11-12.json`	17.3 MB	Dataset maestro con **2,560 muestras** manuscritas reales (entre 74 y 162 por letra), en formato `[[x, y, t], ...]` por sub-trazo.
`models/2d_*.json` y `3d_*.json`	54 arch.	27 modelos 2D y 27 modelos 3D serializados en JSON (`letter`, `n_states`, `n_symbols`, `A`, `B`, `pi`, `expected_length`, `length_std`, `expected_jumps`, `shape_mean`, `shape_std`).
`training/` (3 módulos + `__init__.py`)	551 lín.	`augment.py` (209 lín.): carga dataset, proyecta 2D→S^2 y aplica aumento afín. `train.py` (161 lín.): entrena los 54 modelos con Baum-Welch. `evaluate.py` (181 lín.): validación hold-out en prefijos 30%, 50%, 70%, 100% comparando `K=8` vs `K=16`.
`tests/` (4 módulos)	667 lín.	Suite Pytest con **39 pruebas** de unidad e integración: `test_hmm_engine.py` (188 lín.), `test_feature_extractor.py` (164 lín.), `test_training_pipeline.py` (50 lín.), `test_gui.py` (265 lín.).
`docs/`	5 arch.	`enunciado-proyecto-final-reconocedor-hmm.pdf`, `SPEC.pdf`,

		`AUDITORIA_ARQUITECTURA.pdf`, `SUSTENTACION_MATEMATICA.pdf` y `ecuaciones.md`.

2. Arquitectura, Grafo de Dependencias y Flujo de Datos

Las importaciones forman un **grafo acíclico dirigido (DAG)** sin dependencias circulares y con aislamiento total entre entrenamiento offline, núcleo matemático y GUI:

- `core/HMMEngine.py`: **0 importaciones internas**; solo importa `numpy` y biblioteca estándar (`json`, `dataclasses`, `pathlib`, `typing`).
- `core/config.py`: **0 importaciones internas**; parsea `config/config.toml` con `tomllib`.
- `core/FeatureExtractor.py`: importa únicamente `FeatureConfig, get_config` de `core.config`.
- `gui/RankingPanel.py`: importa `PROJECT_ROOT, get_config` de `core.config` y lee `data/canonical_glyphs.json`.
- `gui/LetterGuide.py`: importa `get_config` de `core.config` y `load_canonical_glyphs` de `gui.RankingPanel`.
- `gui/MainMenu.py`: **0 importaciones internas**; usa solo PySide6.
- `gui/View2D.py` y `gui/View3D.py`: importan `core.config`, su respectivo extractor (`FeatureExtractor2D` o `FeatureExtractor3D`), `HMMClassifier` y `RankingPanel` (además de `pyqtgraph.opengl` en `View3D`).
- `main.py`: importa `core.config`, `FeatureExtractor2D`, `FeatureExtractor3D`, `HMMClassifier` y las 4 vistas de `gui/`. **Nunca importa `training/`.**
- `training/*`: importa `core.config`, `core.FeatureExtractor` y `core.HMMEngine`; nunca es importado en tiempo de ejecución por la GUI.

```
FLUJO OFFLINE: data/glifo-dataset-2026-11-12.json → training/augment.py (load_dataset / map_2d_to_sphere / augment_*)
  → FeatureExtractor2D/3D.extract(realtime=False) → init_segmental_bakis_hmm() → HMM.fit(Baum-Welch + suavizado)
  → exporta 54 archivos JSON en models/2d_*.json y models/3d_*.json

FLUJO EN VIVO: main.py carga models/*.json (solo lectura) → eventos de ratón en View2D / ray-casting en View3D
  → FeatureExtractor2D/3D.extract(..., realtime=True/False) → HMMClassifier.predict_and_project()
  → actualiza RankingPanel (27 probabilidades) + reconstruct_projected_strokes() (curva verde punteada)
```

3. Desglose Completo de Clases, Funciones y Métodos (Líneas Reales Verificadas)

3.1 Raíz y Configuración: `main.py` (165 lín.) y `core/config.py` (141 lín.)

Módulo / Clase	Función / Método y Parámetros	Líneas	Comportamiento Exacto
`main.py` `HMMRecognizerApp`	`__init__(self, models_dir: Path | None=None)`	45–127	Carga `get_config()`, carga `2d_*.json` y `3d_*.json` en `HMMClassifier`, crea barra lateral (`Inicio`, `Modo 2D`, `Modo 3D`, `Guía de letras`) y `QStackedWidget`.
`HMMRecognizerApp`	`navigate_to(self, index: int)`; `show_menu`, `show_2d`, `show_3d`, `show_guia`	129–154	Sincroniza la página activa del `QStackedWidget` y resalta el botón correspondiente; expone atajos `0..3`.
`main.py`	`main()`	157–161	Inicia `QApplication`, abre `HMMRecognizerApp` maximizada (`showMaximized()`) y ejecuta el ciclo de eventos Qt.
`core/config.py`	`FeatureConfig.n_symbols(self)`; `ProjectConfig.file_name(self, letter: str)`	37–39, 53–55	`n_symbols` devuelve `K + n_jump_symbols` (=20); `file_name` mapea `ñ` al alias ASCII `"enie"`.
`core/config.py`	`_validate_letters(letters)`; `_validate_features(f)`	58–87	Valida exactamente 27 minúsculas únicas sin dígrafos ni tildes, $K \in \{8, 16\}$, pasos positivos y ventana impar ≥1.
`core/config.py`	`load_config(path=DEFAULT_CONFIG_PATH)`; `get_config()`	90–141	Lee TOML con `tomllib`, construye `ProjectConfig` validado y lo cachea con `@lru_cache(maxsize=1)`.

3.2 Motor Matemático HMM Nativo: `core/HMMEngine.py` (612 líneas)

Clase / Ámbito	Método / Función y Parámetros	Líneas	Comportamiento Exacto
`HMM`	`__post_init__(self)`; `_refresh_log_matrices(self)`; `_validate_shapes_and_stochastic(self)`	40–74	Convierte `A, B, pi` a `float64`, verifica dimensiones y suma unitaria de filas, y precalcula `log(X + 1e-300)`.
`HMM`	`_huber_quad(z)`; `length_deviation_penalty(self, n_obs: int, complete: bool=False)`	79–106	Penaliza desviaciones respecto a `expected_length` (cuadrática en $\|z\| \le 4$ y continuación lineal C^1 en $\|z\| > 4$; unilateral sobre $\mu + 1.5\sigma$ en parciales).

HMM	shape_penalty(self, shape: np.ndarray | None, n_obs: int, complete: bool=False)	108–135	Penalización gaussiana sobre g ∈ R^7 con peso 1.4 en trazo completo o rampa gradual desde 0.85*μ en parciales.
HMM	jump_penalty(self, observations: Sequence[int], complete: bool=False)	137–152	Penaliza saltos (o ≥ K) en modelos monotrazo (+16/salto) o ausencia de saltos en modelos multitrazo (+24 completo, +12 parcial).
HMM	forward(self, observations: Sequence[int])	157–177	Algoritmo Forward con factores de escala c_t; devuelve (alpha_hat, c, log_prob) donde log_prob = -Σ log(c_t).
HMM	backward(self, observations: Sequence[int], c: np.ndarray)	182–196	Algoritmo Backward usando exactamente los mismos c_t de Forward; devuelve matriz beta_hat (T×N).
HMM	viterbi(self, observations: Sequence[int])	201–225	Programación dinámica en log-espacio con tabla de punteros psi y backtracking; retorna (best_path, best_log_prob).
HMM	fit(self, sequences, max_iter=50, tol=1e-4, pseudo_count=1e-3, shapes=None)	230–334	Baum-Welch (EM) multisecuencia con π fijo y máscara Bakis; aplica suavizado circular 0.78/0.11/0.11 en B[:, :K] y estima expected_length, length_std, expected_jumps, shape_mean, shape_std.
HMM	project_future_symbols(self, current_state, max_symbols=50, spent_in_current=0)	339–380	Estima duración esperada 1/(1-a_jj) por estado (acotada a [1,10]), descuenta pasos consumidos y emite argmax B[j] siguiendo la transición dominante.
HMM	to_dict(self); save_json(self, path); from_dict(cls, data); load_json(cls, path)	385–443	Serialización y deserialización legible en formato JSON sin usar pickle.
Módulo	init_bakis_hmm(letter, n_states, n_symbols)	449–467	Crea HMM con topología Bakis izquierda-derecha (a_ij > 0 solo para j ∈ {i, i+1, i+2}) y π = [1, 0, ..., 0].
Módulo	init_segmental_bakis_hmm(letter, n_states, n_symbols, sequences, pseudo_count=0.1)	470–500	Particiona cada secuencia en N bloques temporales uniformes y estima conteos iniciales con suavizado de Laplace.
HMMClassifier	__init__(self, models=None); add_model(self, hmm); load_from_dir(cls, directory,	513–517, 606–612	Gestiona el diccionario de 27 modelos por letra y los carga automáticamente desde disco.

	`pattern='2d_*.json')`		
`HMMClassifier`	`classify(self, observations, shape=None, complete=False)`	519–570	Suma log-verosimilitud Forward menos penalizaciones y aplica Log-Sum-Exp con τ_{vivo}=`max(1,T/18)` y τ_{final}=`max(1,0.25·T)·max(1,(-S*/T)/1.8)` sobre las 27 clases.
`HMMClassifier`	`predict_and_project(self, observations, max_symbols=45, shape=None, complete=False)`	572–603	Ejecuta `classify`, halla estado actual `q_t*` y pasos consumidos con `viterbi`, y proyecta símbolos futuros del modelo ganador.

3.3 Extractor Geométrico 2D y 3D: `core/FeatureExtractor.py` (582 líneas)

Clase / Ámbito	Método / Función y Parámetros	Líneas	Comportamiento Exacto
Módulo	`compute_shape_features(strokes_2d: Sequence[np.ndarray])`	27–54	Extrae vector global 7D invariante: `[log(w/h), x'_ini, y'_ini, x'_fin, y'_fin, mean_x', mean_y']`.
Dataclasses	`ExtractionResult.to_input_coords(points)`; `ExtractionResult3D.to_world_coords(points_cam)`	58–92	Estructuras de salida; devuelven coordenadas del lienzo 2D (`p = s*p' + p_min`) o coordenadas 3D del mundo (`R * pts @ R_cam`).
`FeatureExtractor`	`__init__(features=None, K=None)`; `n_symbols`; `is_jump(sym)`; `symbol_label(sym)`	98–116	Configura $K \in \{8, 16\}$ y provee utilidades sobre el alfabeto $\|V\|=K+4$ y etiquetas `d0..d15`, `J→↑←↓`.
`FeatureExtractor`	`quantize_direction(angles)`; `quantize_jump(angle)`; `symbol_direction(symbol)`	118–136	Cuantización angular centrada por desplazamiento de medio sector y recuperación del vector unitario `(cos ϕ, sin ϕ)`.
`FeatureExtractor`	`_moving_average(self, points)`	138–147	Promedio móvil simétrico *O(n)* basado en `np.cumsum` que preserva intactos el primer y último punto.
`FeatureExtractor2D`	`normalize(strokes, realtime=False)`; `smooth(points)`; `resample(points)`	153–180	Invierte `Y`, normaliza con piso `250 px` en vivo o `1e-6` final, suaviza y remuestrea cada `D=0.05` mediante `np.interp`.
`FeatureExtractor2D`	`directions(points)`; `jump(end_prev, start_next)`; `extract(strokes, realtime=False)`	182–226	Cuantiza segmentos y saltos (≥`0.05`), concatena símbolos de todos los sub-trazos y adjunta vector 7D.
`FeatureExtractor2D`	`slice_prefix(strokes,`	229–285	Recorta prefijos para evaluación y reconstruye

	`fraction)`; `extract_prefix(...)`; `reconstruct_projected_strokes(...)`		polilíneas proyectadas futuras en coordenadas de pantalla.
`FeatureExtractor3D`	`build_camera_matrix(camera_rotation)`; `rotate_to_camera_frame(strokes, camera_rotation)`	292–328	Construye matriz ortonormal `R_cam ∈ SO(3)` llevando la vista al ecuador frontal `(θ=π/2, ϕ=0)`.
`FeatureExtractor3D`	`cartesian_to_spherical(pts)`; `spherical_to_cartesian(sph)`; `compute_angular_scale(sph, realtime)`	331–354	Conversión `(x,y,z) ↔ (θ, ϕ)` con `np.unwrap` y cálculo de amplitud angular con piso `0.52 rad` en vivo.
`FeatureExtractor3D`	`smooth_spherical(pts)`; `geodesic_distances(pts)`; `resample_geodesic(pts, geodesic_step)`	356–392	Suaviza renormalizando a S^2, mide arcos geodésicos y remuestrea mediante **SLERP** con respaldo lineal si `sin Ω ≤ 1e-8`.
`FeatureExtractor3D`	`directions_spherical(sph)`; `jump_spherical(end_prev, start_next, step)`; `extract(...)`	394–467	Cuantiza rumbo en el plano tangente local `atan2(-dθ, dϕ·sin θ)`, emite saltos esféricos y extrae `ExtractionResult3D`.
`FeatureExtractor3D`	`slice_prefix`; `extract_prefix`; `step_geodesic`; `reconstruct_projected_strokes`; `intersect_ray_sphere`	470–582	Recorta prefijos geodésicos, avanza sobre S^2 con mapa exponencial, devuelve arcos al mundo con `R_cam^T` y resuelve intersección cuadrática rayo-esfera.

3.4 Interfaz Gráfica: Carpeta `gui/` (1,750 líneas)

Archivo / Clase	Métodos y Parámetros	Líneas	Comportamiento Exacto
`MainMenu.py`	`_render_2d_icon(size=72)`; `_render_3d_icon(size=72)`; `_ModeCard`; `MainMenu`	35–198	Genera iconos vectoriales con `QPainter`, tarjetas clicables `_ModeCard` (`__init__` 84–137, `mousePressEvent` 139–142) y pantalla inicial `MainMenu` (148–198).
`View2D.py` `RecognizerCanvas2D`	`__init__`, `resizeEvent`, `clear_strokes`, `undo_last_stroke`, `get_numpy_strokes`, `set_projected_strokes`, eventos ratón, `paintEvent`	26–210	Captura puntos si `d²≥2.25 px`, recentra trazos al redimensionar, soporta deshacer sub-trazo, pinta recuadro 250×250, línea punteada verde de proyección, trazos del usuario y saltos naranja.

View2D.py View2D	__init__, _button_style, _clear_prediction_view, undo_last_stroke, reset_capture, _evaluate_realtime	213–369	Coordina botones (Deshacer trazo, Limpiar lienzo), extrae 2D, actualiza telemetría inferior lbl_obs (x, y, T, proyección, símbolos) y panel lateral.
View3D.py Helpers 3D	_slerp_arc(p0, p1, radius, step=0.02); _densify_spherical; _build_guide_frame_curves	26–101	Densifica segmentos por SLERP sobre la esfera y calcula las 6 curvas del recuadro naranja de ±30° centrado en cámara.
View3D.py SphereCanvas3D	__init__, current_camera_orientation, _init_sphere_scene, _update_guide_frame, _screen_to_sphere_point, clear_strokes, reset_camera_view, set_projected_strokes, _rebuild_*, eventos ratón/rueda	107–369	Renderiza esfera OpenGL con paralelos/meridianos, desproyecta clics con (P·V)^-1 e intersect_ray_sphere, soporta rotación con clic derecho o botón toggle y pinta proyección verde.
View3D.py View3D	__init__, _button_style, _toggle_button_style, _reset_camera_and_canvas, reset_capture, _on_toggle_rotate_mode, _evaluate_realtime	372–545	Coordina barra superior (Centrar esfera, Girar esfera, Limpiar, Reiniciar), extrae sobre active_camera_orientation, actualiza telemetría (θ,ϕ, T) y panel derecho.
RankingPanel.py	load_canonical_glyphs, build_handwritten_glyph_pixmap, _LetterRankRow, RankingPanel (__init__, reset, update_probabilities)	33–283	Renderiza glifos desde canonical_glyphs.json, estiliza scrollbar de 6 px, muestra el ranking estadístico continuo (ΣP=100%) de las 27 filas sin parpadeos.
LetterGuide.py	GuideCanvas (__init__.._draw_direction_arrows); LetterGuideView (__init__, select_letter, _toggle_pause, _on_restart, _cycle_speed)	31–350	Anima el trazo canónico sub-trazo por sub-trazo (ej. 1, 2 o 3 sub-trazos como q con inicio morado) con flechas y círculos numerados, botones a–z, pausa, reinicio y velocidad.

3.5 Entrenamiento Offline y Pruebas: `training/` (551 lín.) y `tests/` (667 lín.)

Archivo	Funciones / Clases y Parámetros	Líneas	Comportamiento Exacto
`training/augment.py`	`canonical_unit_strokes(letter)`; `load_dataset(letter, mode='2d', max_samples=None, dataset_path=None)`	20–86	Carga y cachea plantillas unitarias y muestras limpias de `glifo-dataset-2026-11-12.json` preservando puntos únicos (`i`, `j`); en 3D invoca `map_2d_to_sphere`.
`training/augment.py`	`augment_strokes`; `map_2d_to_sphere`; `augment_strokes_3d`	89–209	Proyecta trazos 2D sobre S^2 y aplica rotación, cizalladura, escala y ruido gaussiano en 2D o en el plano tangente 3D.
`training/train.py`	`_train_letter`, `train_letter_2d`, `train_letter_3d`, `_train_all`, `train_all_2d`, `train_all_3d`, `main`	21–157	Núcleo unificado de entrenamiento: aumenta letras con <80 muestras, inicializa por segmentación, ejecuta `HMM.fit` (`pseudo_count=1e-2`, `max_iter=30`) y exporta 54 JSON.
`training/evaluate.py`	`EvaluationReport`; `_evaluate_k_mode`, `evaluate_k_2d`, `evaluate_k_3d`, `compare_k_configs`	26–176	División estratificada 75%/25% y cálculo de accuracy Top-1/Top-3 en prefijos 30%, 50%, 70%, 100% comparando `K=8` frente a `K=16`.
`tests/` (4 archivos)	`test_hmm_engine.py` (12), `test_feature_extractor.py` (17 casos), `test_training_pipeline.py` (3), `test_gui.py` (7)	188, 164, 50, 265	39 pruebas automatizadas verificando estocasticidad, Forward/Viterbi/Backward, penalización Huberizada C^1, calibración en garabatos extremos, SLERP, botones 2D, telemetría en vivo y guía modal.